REACT · DERS MATERYALI

API, Axios ve Custom Hook

json-server ile sahte API, Axios ile istek atma ve useFetch hook'u

```
GET /todos
// 200 OK
[
  { "id": 3, "title": "Kitap oku" },
  { "id": 4, "title": "Api çalış" }
]
```

PLAN

Bugün neler öğreneceğiz?

01

API ve json-server

API, HTTP temelleri ve sahte API kurulumu

02

Axios

fetch ile farkı, instance, params, interceptor

03

Todo App

Listeleme, ekleme ve silme (CRUD) akışı

04

Custom Hook

Tekrar eden kodu useFetch ile tek yerde toplamak

01

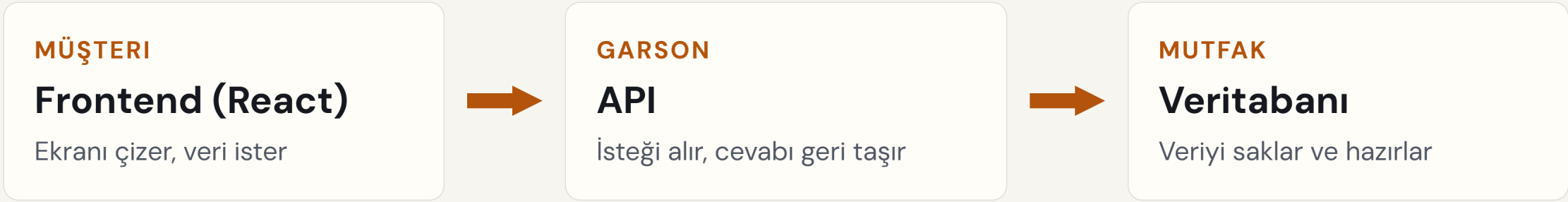
API ve json-server

Veri nereden gelir? Sahte bir API'ı dakikalar içinde ayağa kaldıralım.

01 · API VE JSON-SERVER

API nedir?

API (Application Programming Interface), bir uygulamanın başka bir uygulamayla konuşmasını sağlayan arayüzdür.



Frontend veritabanına doğrudan ulaşmaz; ekranda göstereceği dinamik veriyi API'a istek atarak alır.

İstek ve yanıt nasıl görünür?

```
// İSTEK
POST /todos
Content-Type: application/json

{ "title": "Kitap oku",
  "category": "iş" }
```

- **Metot:** ne yapılacak (GET, POST…)
- **Adres (URL):** hangi kaynak
- **Header:** ek bilgi (içerik tipi vb.)
- **Body:** gönderilen veri

```
// YANIT
201 Created
Content-Type: application/json

{ "id": 19, "title": "Kitap oku",
  "category": "iş" }
```

- **Durum kodu:** sonuç (200, 404…)
- **Header:** ek bilgi
- **Body:** dönen veri, genelde JSON

HTTP metotları ve CRUD

Metot	CRUD	Ne yapar?
GET	Read (Oku)	Veriyi getirir
POST	Create (Oluştur)	Yeni kayıt ekler
PUT	Update (Güncelle)	Kaydın tamamını değiştirir
PATCH	Update (Güncelle)	Kaydın yalnızca bir kısmını değiştirir
DELETE	Delete (Sil)	Kaydı kaldırır

Aynı adrese (/todos) farklı metotla istek atmak farklı bir işlem yaptırır.

Durum kodları

Kod	Anlamı	Ne zaman döner?
200	OK	İstek başarılı (GET, DELETE)
201	Created	Yeni kayıt oluşturuldu (POST)
400	Bad Request	İstek hatalı veya eksik
404	Not Found	Adres veya kayıt bulunamadı
500	Server Error	Sunucu tarafında hata oluştu

2xx · başarılı 4xx · istemci hatası 5xx · sunucu hatası

01 · API VE JSON-SERVER

json-server nedir?

Hızlıca **REST API** oluşturmaya yarayan bir JavaScript kütüphanesi

Gerçek veritabanı yerine **db.json** dosyasını kullanır

Mock (sahte) API: backend hazır olmadan frontend geliştirmeye başlayabilirsin

```
// db.json
{
  "todos": [
    {
      "id": 3,
      "title": "Kitap oku",
      "category": "finans",
      "date": 1736035200000
    }
  ]
}
```

01 · API VE JSON-SERVER

json-server kurulumu

1	Kütüphaneyi projeye indir	`npm i json-server@0.17.4`
2	Ana klasöre veritabanı dosyasını oluştur	`db.json`
3	API'ı ayağa kaldır	`npx json-server --watch db.json --port 4000`
4	Komutu package.json'a script yap	`"server": "json-server --watch db.json --port 4000"`
5	Bir terminalde API'ı başlat	`npm run server`
6	Yeni bir terminalde frontend'i başlat	`npm run dev`

01 · API VE JSON-SERVER

Otomatik oluşan adresler

db.json içindeki todos dizisi için json-server şu adresleri hazır verir. Taban adres: **http://localhost:4000**

İstek	Sonuç
GET /todos	Tüm todo'ları listeler
GET /todos/3	id'si 3 olan todo'yu getirir
POST /todos	Yeni todo ekler (id'yi sunucu üretir)
PUT /todos/3	3 numaralı todo'yu tamamen değiştirir
PATCH /todos/3	3 numaralı todo'nun bazı alanlarını günceller
DELETE /todos/3	3 numaralı todo'yu siler

Sorgu parametreleri

Parametre	Ne yapar?	Örnek adres
_sort, _order	Sıralar	/todos?_sort=date&_order=desc
category	Alana göre filtreler	/todos?category=finans
_limit	Sayıyı sınırlar	/todos?_limit=5
_page	Sayfalama yapar	/todos?_page=2&_limit=5
q	Metin araması yapar	/todos?q=api

Uygulamada yeni eklenen todo en üstte görünsün diye **date** alanına göre azalan sırada istiyoruz:
_sort=date&_order=desc

02

Axios

fetch ile aynı işi daha az kodla ve daha fazla kontrolle yapan HTTP istemcisi.

02 · AXIOS

Neden Axios?

Pratik

fetch'e göre daha az kod yazarız.

Otomatik JSON

İstek ve yanıt verisini kendisi JSON ile JS arasında çevirir.

Header'lar hazır

Gerekli bazı header'ları otomatik ekler.

Interceptor

Her istekte veya yanıtta araya girip fonksiyon çalıştırabiliriz.

Varsayılan ayarlar

baseURL, timeout, headers ve params bir kez tanımlanır.

Kurulum

```
npm i axios
```

02 · AXIOS

GET: veri okuma

FETCH

```
fetch("http://localhost:4000/todos")
  .then((res) ⇒ res.json())
  .then((data) ⇒ setTodos(data))
  .catch((err) ⇒ setError(err.message));
```

AXIOS

```
api
  .get("/todos")
  .then((res) ⇒ setTodos(res.data))
  .catch((err) ⇒ setError(err.message));
```

Axios'ta cevabın verisi **res.data** içindedir; **res.json()** çağırmaya gerek yoktur.

02 · AXIOS

POST: veri ekleme

FETCH

```
fetch("http://localhost:4000/todos", {
  method: "POST",
  body: JSON.stringify(newTodo),
  headers: {
    "Content-Type": "application/json",
  },
})
  .then((res) => res.json())
  .then((data) => addTodo(data));
```

AXIOS

```
api
  .post("/todos", newTodo)
  .then((res) => addTodo(res.data));
```

Gönderilen nesneyi **JSON.stringify** ile çevirmeye ve **Content-Type** header'ı yazmaya gerek yok.

02 · AXIOS

DELETE: veri silme

FETCH

```
fetch(`http://localhost:4000/todos/${id}`, {
  method: "DELETE",
})
  .then(() => deleteTodo(id))
  .catch(() => alert("silme başarısız"));
```

AXIOS

```
api
  .delete(`/todos/${id}`)
  .then(() => deleteTodo(id))
  .catch(() => alert("silme başarısız"));
```

İstek başarılı olunca **state'i de güncelle** (deleteTodo); yoksa ekranda eski liste kalır.

02 · AXIOS

Hata yönetimi

```
api
  .get("/todos")
  .catch((err) ⇒ {
    if (err.response) {
      // sunucu cevap verdi: 404, 500
    } else if (err.request) {
      // cevap gelmedi: sunucu kapalı
    } else {
      // istek hazırlanırken hata
    }
  });
```

fetch

404 ve 500 gibi yanıtlar catch'e düşmez; res.ok ile elle kontrol gerekir.

axios

2xx dışındaki her durum kodu otomatik catch'e düşer.

err.message

Kısa hata metni; ekranda göstermek için yeterli.

axios.create: tek yerde ayar

```
// src/api/index.js
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:4000",
  timeout: 3000,
  // params: { lang: "tr" },
  // headers: { Authorization: "..." },
});

export default api;
```

baseURL: temel adres, her isteğin başına eklenir

timeout: milisaniye cinsinden zaman aşımı (3000 = 3 sn)

params: her isteğe eklenen varsayılan parametreler

headers: her isteğe eklenen varsayılan header'lar

02 · AXIOS

params ile sorgu parametreleri

```
const params = { _sort: "date", _order: "desc" };

api.get("/todos", { params })
```



```
// giden istek
GET http://localhost:4000/todos?_sort=date&_order=desc
```

Adresi elle birleştirmek yerine nesne yazarız.

Özel karakterleri axios kodlar.

Dinamik değerler (arama, sayfa) kolay eklenir.

02 · AXIOS

PUT ve PATCH: veri güncelleme

PUT · TAMAMINI DEĞİŞTIRIR

```
api
  .put(`/todos/${id}`, updatedTodo)
  .then((res) ⇒ updateTodo(res.data));
```

PATCH · YALNIZCA GÖNDERILEN ALAN

```
api
  .patch(`/todos/${id}`, { title })
  .then((res) ⇒ updateTodo(res.data));
```

APP.JSX · STATE'I GÜNCELLE

```
const updateTodo = (updated) ⇒ {
  setTodos(todos.map((t) ⇒ (t.id ⩵ updated.id ? updated : t)));
};
```

Interceptor: araya girmek

```
// her istekten önce çalışır
api.interceptors.request.use((config) => {
  config.headers.Authorization = "Bearer token";
  return config;
});

// her yanıttan sonra çalışır
api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) alert("Oturum bitti");
    return Promise.reject(err);
  }
);
```

Request

İstek gitmeden önce yakalanır; token gibi ortak bilgi eklemek için idealdir.

Response

Yanıt bileşene ulaşmadan yakalanır; hataları tek yerde yönetiriz.

03

Todo App

Öğrendiklerimizi bir CRUD uygulamasında birleştirelim:
listele, ekle, sil.

Uygulamanın yapısı

```
src/
  api/index.js          // axios örneği
  App.jsx               // state + liste
  todo-components/
    TodoForm.jsx        // ekleme
    ListItem.jsx        // silme
```

App.jsx

todos, isLoading ve error state'leri burada; addTodo ve deleteTodo burada tanımlı.

TodoForm.jsx

API'a POST atar, sonucu addTodo ile App'e bildirir.

ListItem.jsx

API'a DELETE atar, sonucu deleteTodo ile App'e bildirir.

Listeleme: sayfa açılınca GET

```
const [isLoading, setIsLoading] = useState(true);
const [error, setError] = useState(null);
const [todos, setTodos] = useState(null);

useEffect(() ⇒ {
  const params = { _sort: "date", _order: "desc" };

  api
    .get("/todos", { params })
    .then((res) ⇒ setTodos(res.data))
    .catch((err) ⇒ setError(err.message))
    .finally(() ⇒ setIsLoading(false));
}, []);
```

3 state: yükleniyor mu, hata var mı, veri ne?

[ ] bağımlılık dizisi boş: istek yalnızca ilk render'da atılır.

finally: başarılı da olsa hatalı da olsa yükleniyor durumu biter.

Üç durum, üç ekran

```
{isLoading ? (
  <h1>Yükleniyor..</h1>
) : error ? (
  <h1>Hata!</h1>
) : (
  todos.map((todo) => (
    <ListItem key={todo.id} todo={todo} deleteTodo={deleteTodo} />
  ))
)}
```

isLoading ise yükleniyor yazısı

error varsa hata mesajı

Aksi halde **liste**; her elemana benzersiz key verilir.

Ekleme: form gönderilince POST

```
const handleSubmit = (e) ⇒ {
  e.preventDefault();

  const title = e.target[0].value;
  const category = e.target[1].value;

  const newTodo = { title, category, date: Date.now() };

  api
    .post("/todos", newTodo)
    .then((res) ⇒ addTodo(res.data))
    .catch(() ⇒ alert("İşlem başarısız oldu"));

  e.target.reset();
};
```

preventDefault: form gönderilince sayfa yenilenmesin.

Date.now(): milisaniye cinsinden tarih; sıralamada kullanılır.

id: sunucu üretir; res.data içinde id'li kayıt döner.

addTodo: yeni kaydı listenin başına ekler.

Silme: önce API, sonra state

LISTITEM.JSX

```
const handleClick = () ⇒ {
  if (!confirm("Silmek istediğinizden emin misiniz?")) return;

  api
    .delete(`/todos/${todo.id}`)
    .then(() ⇒ deleteTodo(todo.id))
    .catch(() ⇒ alert("silme işlemi başarısız"));
};
```

APP.JSX

```
const deleteTodo = (deleteId) ⇒ {
  setTodos(todos.filter((todo) ⇒ todo.id ⇍⇎ deleteId));
};
```

04

Custom Hook

Her bileşende kopyaladığımız veri çekme mantığını tek bir yerde toplayalım.

04 · CUSTOM HOOK

Problem: aynı kodu tekrar yazıyoruz

PRODUCTLISTONE.JSX

```
const [data, setData] = useState(null);
const [error, setError] = useState(null);
const [isLoading, setIsLoading] = useState(true);
useEffect(() => {
  setIsLoading(true);
  fetch("https://dummyjson.com/products")
    .then((res) => res.json())
    .then((data) => setData(data.products))
    .catch((err) => setError(err.message))
    .finally(() => setIsLoading(false));
}, []);
```

USERLISTONE.JSX

```
const [data, setData] = useState(null);
const [error, setError] = useState(null);
const [isLoading, setIsLoading] = useState(true);
useEffect(() => {
  setIsLoading(true);
  fetch("https://dummyjson.com/users")
    .then((res) => res.json())
    .then((data) => setData(data.users))
    .catch((err) => setError(err.message))
    .finally(() => setIsLoading(false));
}, []);
```

Mavi: farklı olan kısım. Geri kalan her şey birebir aynı.

04 · CUSTOM HOOK

Custom hook nedir?

Kural

Adı **use** ile başlayan bir fonksiyondur.

Yetenek

İçinde useState, useEffect gibi diğer hook'ları kullanabilir.

Amaç

Tekrar eden mantığı tek yerde toplayıp birçok bileşende kullanmak.

Dikkat

Mantık paylaşılır, state paylaşılmaz: her çağrı kendi state'ini oluşturur.

04 · CUSTOM HOOK

useFetch hook’u

```
// src/hooks/useFetch.js
const useFetch = (url) => {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    setIsLoading(true);

    fetch(url)
      .then((res) => res.json())
      .then((data) => setData(data))
      .catch((err) => setError(err.message))
      .finally(() => setIsLoading(false));
  }, []);

  return { isLoading, error, data };
};
```

Girdi: url

Farklı adreslerle çağrılabilir; adres hook’un dışından gelir.

Çıktı: nesne

isLoading, error ve data döner; bileşen ihtiyacı olanı alır.

İçerik: bildiğimiz kod

Üç state ve useEffect; sadece bir fonksiyonun içine taşındı.

04 · CUSTOM HOOK

Bileşende kullanımı

```
const ProductListTwo = () ⇒ {
  const { data, isLoading, error } = useFetch(
    "https://dummyjson.com/products"
  );

  if (isLoading) return <h1>Yükleniyor...</h1>;
  if (error) return <h1>Hata!: {error}</h1>;

  return data.products.map((p) ⇒ <p key={p.id}>{p.title}</p>);
};
```

1 satır

3 state, useEffect ve fetch zinciri yerine tek hook çağrısı.

Her bileşen kendi state'ine sahip; **ProductListTwo** ve **UserListTwo** aynı hook'u çağırsa da veriler karışmaz.

04 · CUSTOM HOOK

Hook kuralları

1. Üst seviyede çağır

if, döngü veya iç içe fonksiyon içinde hook çağrılmaz.

2. Yalnızca bileşen veya hook içinde

Sıradan JavaScript fonksiyonlarında kullanılmaz.

3. Her render'da aynı sırada

Bu yüzden erken return'den önce çağrılmalıdır.

```
// Yanlış: hook, return'den sonra
if (!isOpen) return null;
const [text, setText] = useState("");

// Doğru: hook en üstte
const [text, setText] = useState("");
if (!isOpen) return null;
```

04 · CUSTOM HOOK

useFetch'i geliştirelim

```
const useFetch = (url, params) => {
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    setIsLoading(true);
    setError(null);
    api
      .get(url, { params })
      .then((res) => setData(res.data))
      .catch((err) => setError(err.message))
      .finally(() => setIsLoading(false));
  }, [url]);

  return { data, error, isLoading };
};
```

[ ] yerine [url]

Adres değişince hook yeniden istek atar.

fetch yerine api

baseURL ve timeout hazır; veri res.data içinde; params desteklenir.

params uyarısı

Nesne her render'da yeniden oluşur; bağımlılık dizisine doğrudan koyma.

04 · CUSTOM HOOK

Sık yapılan hatalar

{error.message}

error zaten bir string; ekranda boş görünür. Doğrusu {error}.

key eksik

map içinde key yoksa React uyarı verir; key={item.id} ekle.

Boş bağımlılık dizisi

useFetch'te [ ] varsa url değişince yeni istek atılmaz; [url] yaz.

Çift DELETE isteği

ListItem'da fetch ve axios birlikte duruyor; karşılaştırma içindi, tek yöntem bırak.

product.name

dummyjson ürünlerinde alan adı title'dır; name yazınca boş görünür.

Hata ayıklarken

Önce konsola ve Network sekmesine bak, sonra state'in tipini kontrol et.

ÖZET

Bugün öğrendiklerimiz

API ve json-server

API istek ve yanıttan oluşur. json-server, db.json ile sahte API kurar.

Axios

axios.create ile tek yerde ayar. Veri res.data içinde, params nesne olarak gider.

Todo App

Üç state ile GET. POST ve DELETE başarılı olunca state güncellenir.

Custom Hook

use ile başlar, hook'ları kullanır. Mantık paylaşılır, state paylaşılmaz.

ALIŞTIRMA

Sıra sizde

1

Güncelle

Todo başlığını düzenle: api.patch ile gönder, state'i map ile güncelle.

2

Filtrele

Kategoriye göre filtrele: seçilen kategoriyi params ile API'a gönder.

3

Kendi hook'un

useFetch'i axios ve [url] ile yaz, todo listesini onunla çek. Bonus: refetch döndür.